User Management Dashboard - Project Report

Introduction

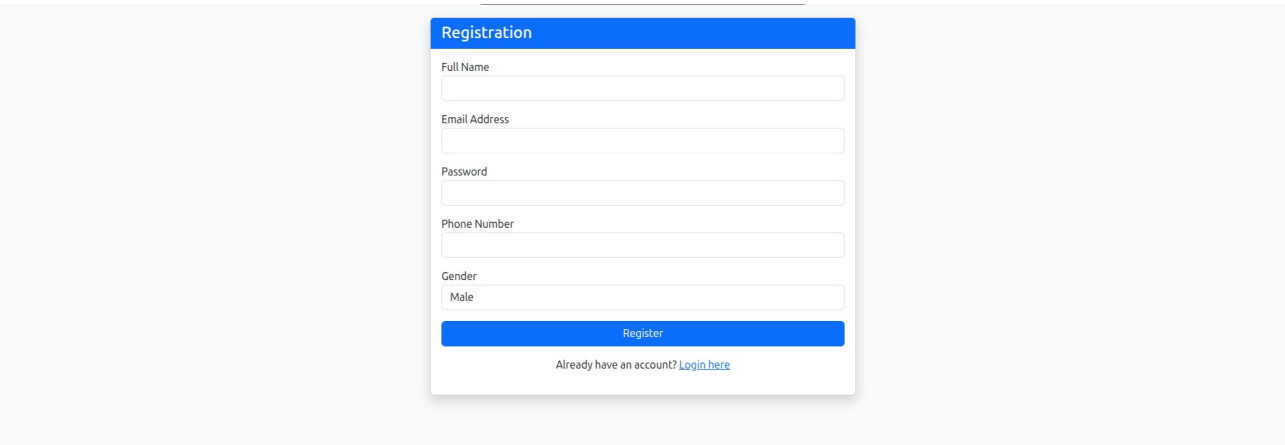
Welcome to the User Management Dashboard! This project is a complete web application that makes managing users simple and secure. Built with Python Flask and MySQL, it's designed to be both powerful and easy to use, whether you're learning web development or building a real-world application.

Key Features Explained

1. User Registration

When someone signs up, they provide basic information like their name, email, phone number, gender, and password. The system takes care of security by:

1. Hashing passwords before storing them (no plain text passwords!)
2. Checking if an email is already registered to prevent duplicates
3. Validating all inputs before creating the account



Behind the scenes:

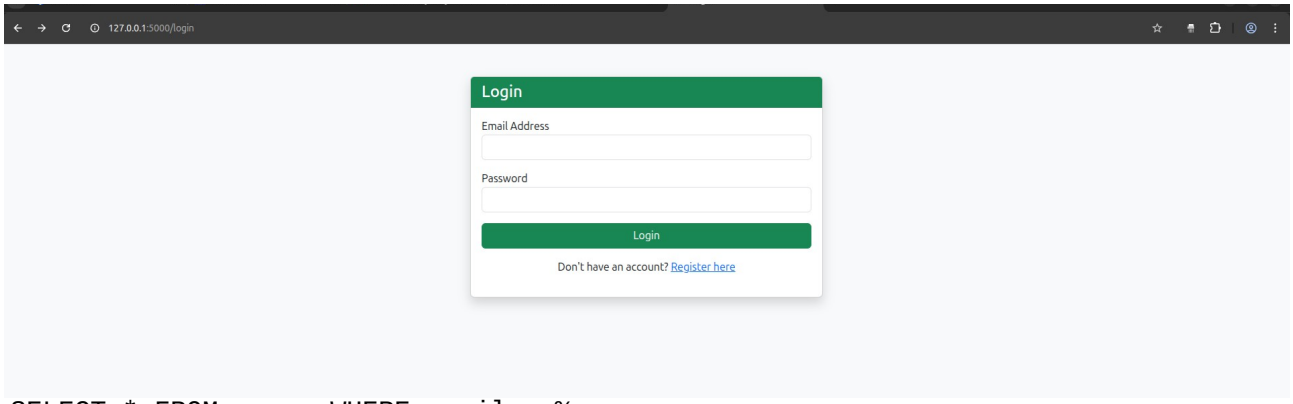
```
-- Check if email exists
SELECT * FROM users WHERE email = %s;

-- Create new user
INSERT INTO users (name, email, password, phone, gender)
VALUES (%s, %s, %s, %s, %s);
```

2. Secure Login System

Users log in with their email and password. The system verifies credentials and creates a secure session that remembers who's logged in. This means users don't have to log in again on every page.

Session management includes:

1. User ID stored securely
2. User name for personalization
3. Automatic redirect to dashboard after successful login




```
SELECT * FROM users WHERE email = %s;
```

3. Dynamic Dashboard

The dashboard is where everything comes together. Users see:

Personal Profile Section:

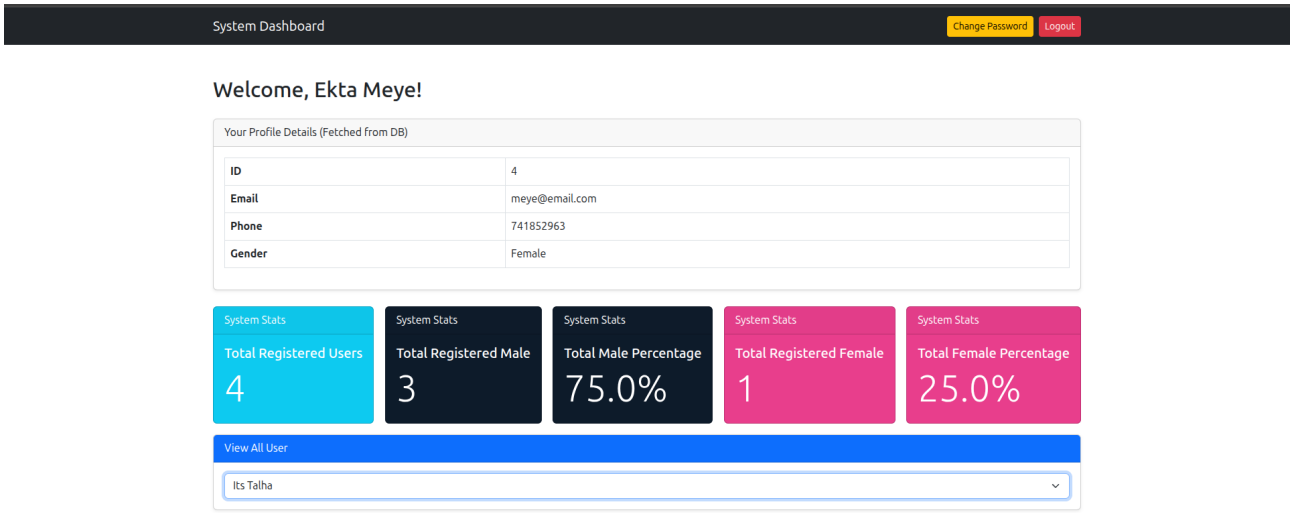
1. Their unique ID
2. Email address
3. Phone number
4. Gender

System Statistics:

1. Total number of registered users
2. Male and female user counts
3. Gender distribution percentages
4. Visual cards with color-coded statistics

Interactive User Directory:

1. Dropdown menu showing all users
2. Click any name to instantly view their details
3. No page reload needed - it's all dynamic!



Database queries powering the dashboard:

```
-- Get specific user details
SELECT * FROM users WHERE id = %s;

-- Count all users
SELECT COUNT(*) as count FROM users;

-- Count by gender
SELECT COUNT(*) as count FROM users WHERE gender='Male';
SELECT COUNT(*) as count FROM users WHERE gender='Female';

-- Get user list for dropdown
SELECT id, name FROM users ORDER BY name ASC;
```

4. Password Change Feature

Security is a priority, so changing passwords requires verification. Here's how it works:

Step 1: User enters their current password

- If wrong → Session ends and user is logged out (security measure)
- If correct → Proceed to step 2

Step 2: User enters and confirms their new password

- Password is hashed before saving
- User stays logged in with updated credentials

```
-- Verify current password
SELECT password FROM users WHERE id = %s;

-- Update to new password
UPDATE users SET password = %s WHERE id = %s;
```

5. Logout

Simple but important - users can securely end their session, which clears all stored information and redirects them to the login page.

Database Architecture

Database: `user_system`

The heart of the application is a clean, well-structured database:

Users Table Structure

Column	Type	Description
id	INT (Primary Key)	Unique identifier, auto-incremented
name	VARCHAR(100)	User's full name
email	VARCHAR(100)	Unique email address
password	VARCHAR(255)	Hashed password (never plain text!)
phone	VARCHAR(20)	Contact number
gender	ENUM('Male','Female')	User's gender

Setup Script:

```
CREATE DATABASE user_system;
USE user_system;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    gender ENUM('Male','Female') NOT NULL
);
```

How the Backend Works

Application Routes

The Flask application is organized into clear, logical routes:

- **`/register`** - Handles new user signups
- **`/login`** - Processes login attempts and creates sessions
- **`/dashboard`** - Shows the main interface with stats and user data
- **`/change_password`** - Manages password updates
- **`/logout`** - Ends user sessions

Session Management

Sessions keep track of logged-in users across pages. When you log in, the system stores:

- Your user ID
- Your name for personalization

This prevents unauthorized access to protected pages like the dashboard.

Security Measures

Password Security:

- All passwords are hashed using Werkzeug's security functions
- Original passwords are never stored or logged

Session Security:

- Failed password verification automatically logs users out
- Sessions expire properly on logout

SQL Injection Prevention:

- All database queries use parameterized inputs (`%s`)
- User input is never directly inserted into queries

Frontend Design

The user interface uses **Bootstrap 5** for a modern, responsive design that works on any device.

Design Elements:

- Clean navigation bar with user actions
- Color-coded statistic cards (blue, pink, green, yellow)
- Responsive grid layout
- Interactive dropdown menus
- Forms with proper validation

Quick Reference: SQL Queries

Purpose	Query
Check existing email	`SELECT * FROM users WHERE email = %s`
Register new user	`INSERT INTO users (name, email, password, phone, gender) VALUES (...)`
Login verification	`SELECT * FROM users WHERE email = %s`
Get all users count	`SELECT COUNT(*) as count FROM users`
Count male users	`SELECT COUNT(*) as count FROM users WHERE gender='Male'`
Count female users	`SELECT COUNT(*) as count FROM users WHERE gender='Female'`
Get users for dropdown	`SELECT id, name FROM users ORDER BY name ASC`
Update password	`UPDATE users SET password = %s WHERE id = %s`

Future Enhancements

Want to take this project further? Here are some ideas:

User Management

- **Role-based access** - Add admin and regular user roles

- **Edit user profiles** - Let users update their information
- **Delete accounts** - Allow account removal with confirmation
- **User search** - Find users by name, email, or other criteria

Security

- **Email verification** - Confirm email addresses on signup
- **Password reset** - Forgot password functionality via email
- **Two-factor authentication** - Add an extra security layer
- **Password strength meter** - Visual feedback on password quality

Conclusion

This User Management Dashboard is a complete, production-ready application that demonstrates modern web development practices. It successfully combines backend logic, database management, and frontend design into a cohesive, secure, and user-friendly system.